



Netflix's data usage control

Netflix finally introduces a data usage control which will be a boon for metered connections all over the world. <http://dgit.in/Netflix3>



SpaceX plans Mars trip

SpaceX plans to send its Dragon spacecraft to Mars by 2018. <http://dgit.in/SpaceXMars>

Digit Squad

of files being uploaded. Don't go with excluding specific extensions instead go on by just allowing specific extensions that you expect the user to use, for all the others, just throw up an error. Even by mistake, do not transfer the unwanted file to your server and then throw the error, that's just stupid but often practiced. Make sure that all these checks are performed server-side because if you are doing it on the user-end then the bad guy can easily dodge the check (hint: Inspect element). Lastly, the least effective advice, use 'chmod' and never set any uploaded file's permission to 777 (that means read, write, execute access to everyone), you must use 644 or whatsoever your application demands.

Validation of requests being made

A few months back, a person found a vulnerability on YouTube. All he did was modifying the POST request, from his own Video ID to the target video's ID, that allowed him to delete any YouTube video without the uploader's consent which could have actually created a big mess. Getting back to our dummy website, if the user clicks on 'delete profile', a POST request will be generated which can be tampered easily by using plugins such as 'Tamper data' or 'Postman' (for Firefox and Chrome respectively). This is how the bad guy will proceed: He'll turn on the plugin, click on Delete Profile on the user dashboard, interfere with the POST request being made using plugin and change the ID from his own to someone else's ID and then let the request proceed. And as you might have guessed, someone else's account will get deleted without that user's consent.

This is why it's extremely important to verify whether the action being executed is legitimately initiated or not. To overcome such requests, you'd need to implement Cross Site Request Forgery (CSRF). When the user logs in, generate some random values and stick it up to the user's session and have a copy of it on the server side too. For every requests, insert this random value in the POST data (use hidden field for forms) and cross-check this value, whether it matches with the one stored on server side or not. If it doesn't, simply return an error, and that's how you made your whole environment more secure.

SQL Injection

You probably heard this term earlier is your website safe from SQL Injections? This is probably the least effort consuming way of hacking into any poorly coded website and of course, tons of websites (especially small businesses) are vulnerable to SQL injection. Imagine you are on the login page of our dummy website, a normal user will input his username and the password for it, and if they match, the user will be logged in. However, what if instead of a normal user, a bad guy comes up and he inputs some cryptic strings, guess what, he will get logged into the first account that's present in the database, which generally is of an administrator.

What actually happened over here is that the cryptic string altered the behaviour of your SQL commands. In any code, 'terminators' play important role and that cryptic string, consisting of numbers and conditions, basically tricked the program. So, if our MySQL query was something like:

```
SELECT id FROM users WHERE username = $username AND password = $pwd;
```

And the bad guy puts username as "1 OR 1 = 1; --" and submits the form. This will actually convert our MySQL statement to
`SELECT id FROM users WHERE username = 1 OR 1 = 1; -- AND password = any_random_text;`
Which will get the user logged into someone's account without even knowing the password. (Double dashes above stands for comment in SQL).

To overcome such vulnerabilities, you should use "mysql_real_escape_string()" function (for PHP version < 5.0) or start using 'mysqli' (for PHP version > 5.0). Both of them will filter out the unwanted characters and no longer the bad guy could exploit your site.

With those three practises you have actually made your website much more secure and are assured to keep script kiddies at bay. And wait, there still lies a lot of undiscovered vulnerabilities that keep popping out every day thus making none of the systems secure enough. So keep an eye on the news and efficiently improve your code wherever possible. 🚩

Disclaimer: The above topic is purely for educational purposes. Do keep in mind that you are fiddling with the website's coding and any erroneous code can render your website dysfunctional. A first hand experience of writing code and building a website is essential. Digit and its parent company are not legally responsible for any outcome that may arise out of the implementation of the above article in any and all ways.



PC, Laptop
Tablet, Mobile

सुरक्षा



92.72.70.70.50
98.22.88.25.62

india
antivirus.com